MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Empowered Autonomous)

NEW PANVEL

PROJECT REPORT ON

"NEXUS: A REAL-TIME CHAT PLATFORM"

IN PARTIAL FULFILLMENT OF
M.Sc. DATA ANALYTICS
SEMESTER I 2026-27

PROJECT GUIDE
Prof. Omkar Sherkhane

SUBMITTED BY: **Razi Ahmed Mumtaz**
ROLL NO: **2857**

Mahatma Education Society's

PILLAI COLLEGE OF ARTS,COMMERCE & SCIENCE

(Empowered Autonomous)

Re-accredited "A" Grade by NAAC (3rd Cycle)

Project Completion Certificate

THIS IS TO CERTIFY THAT

RAZI AHMED MUMTAZ

of **M.Sc. Data Analytics Part - I** has completed the project titled **"Nexus: A Real-Time Chat Platform"** of subject **Next Generation Databases** under our guidance and supervision during the academic year 2026-27 in the department of Computer Science.

Project Guide	Course Coordinator	Head of the Department

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

INDEX

Sr. No.	Topic
1	Cover Page
2	Project Completion Certificate
3	Introduction (About the Project)
4	Tools and Techniques Used
5	Code and Output Screenshots
6	Conclusion

3. INTRODUCTION

3.1 About the project

Nexus is a full-stack, real-time chat platform that combines the two most popular styles of online communication:

- **WhatsApp style:** private one-to-one chats and group chats, with read receipts (single / double / blue ticks), typing indicators, online status and "last seen".
- **Discord style:** communities called **servers**, each with several text **channels**, invite links, roles (Owner, Admin, Member) and an audit log of admin actions.

Messages are delivered instantly over a **WebSocket** connection, stored permanently in **MongoDB** (a NoSQL document database), and fast-changing counters such as online users, unread badges and rate limits are kept in **Redis** (an in-memory key-value database). The project is deployed free on the cloud and can be opened on a laptop or a phone.

3.2 Problem statement

Students and teams usually use one app for private chats and a different app for communities. Building a chat system is also a good example of a **Next Generation Database** problem: the data is semi-structured (messages, reactions, replies, mentions), grows very fast, must be read in "newest first" pages, and many small counters change on every action. A rigid relational schema and a single database are not the best fit for all of these needs.

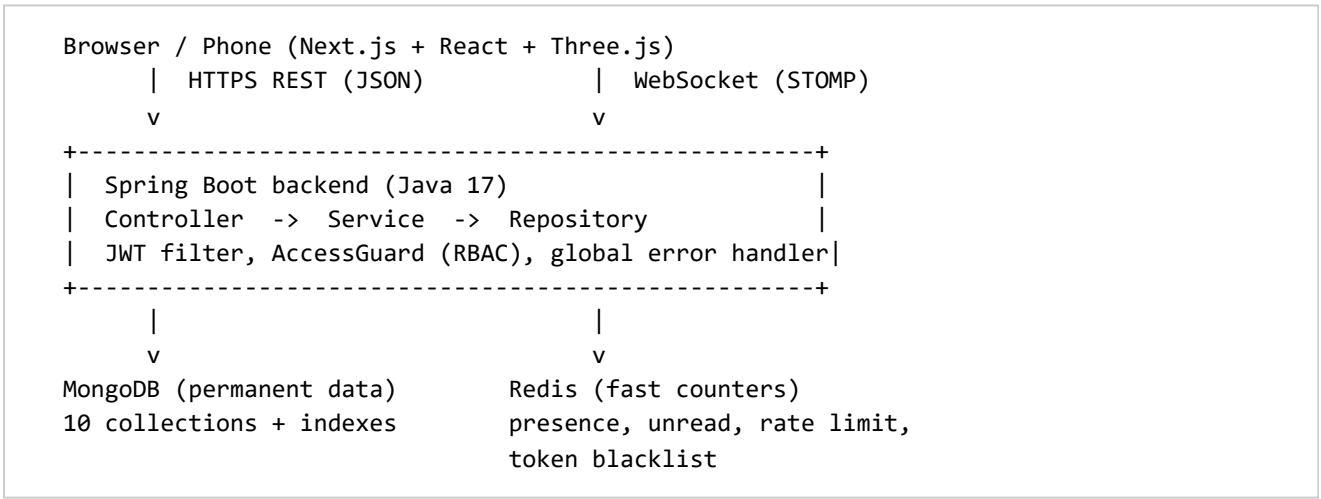
3.3 Objectives

1. Design a document data model in MongoDB with **10 collections** and the right **indexes** for chat queries.
2. Use Redis for **presence, unread counters, rate limiting and token blacklisting**, with a safe fallback when Redis is down.
3. Deliver messages in real time with **WebSocket + STOMP**, without duplicates even when the network retries (**idempotency**).
4. Secure the system with **BCrypt** password hashing, **JWT** login and **role-based access control (RBAC)**.
5. Handle every error in one consistent format in both backend and frontend (custom exceptions + error codes).
6. Build a modern, mobile-responsive 3D user interface and deploy everything on free cloud services.

3.4 Main features

Module	Features
Authentication	Register, login, logout, JWT valid 72 hours, login rate limit (10/min), logout blacklists the token.
Private & group chats	One DM per pair of users, groups with owner/admin/member, reply, react, edit, soft delete, pin, search, @mentions.
Read receipts	Clock → single grey tick (saved) → double ticks (read by some) → blue ticks (read by all), computed from each member's `lastReadAt`.
Servers & channels	Create server (gets #general, #announcements, #off-topic), invite links with expiry and max uses, discover public servers, change roles, kick, audit log.
Social	Friend requests (accept / decline / remove), live notifications with a bell badge.
Realtime	Instant messages, typing indicator, online/offline presence, unread badges, all over one WebSocket.
UI	Next.js + React with Three.js 3D scenes, dark futuristic theme, mobile bottom navigation.

3.5 System overview



3.6 Scope

The project covers the complete flow of a messaging product: accounts, conversations, communities, realtime delivery, permissions, storage design, error handling, automated testing and cloud deployment. Voice/video calls and end-to-end encryption are outside the current scope and are listed as future work in the conclusion.

4. TOOLS AND TECHNIQUES USED

4.1 Tools (technology stack)

Layer	Tool / Technology	Why it was used
Database (main)	**MongoDB** (NoSQL, document)	Flexible JSON-like documents for messages with reactions, mentions and replies; compound indexes; horizontal scaling. Hosted on MongoDB Atlas.
Database (cache)	**Redis** (NoSQL, key-value, in-memory)	Atomic counters (HINCRBY, INCR) with expiry (TTL) for presence, unread badges, rate limits and logout blacklist. Hosted on Upstash.
Backend	Java 17, **Spring Boot 3.5**	REST APIs, dependency injection, Spring Data MongoDB / Redis, validation.
Realtime	WebSocket + **STOMP** (Spring Messaging)	Server pushes messages, typing, presence and notifications instantly.
Security	Spring Security, **JWT** (jjwt), **BCrypt**	Stateless login tokens, hashed passwords, per-request authentication.
Frontend	**Next.js 16**, React 19, TypeScript, Tailwind CSS 4	Fast pages, typed code, responsive design.
3D graphics	**Three.js** via React Three Fiber	Futuristic animated 3D scenes on landing and login pages.
Forms	Formik + Yup	Form state and validation with field-level error messages.
Realtime client	@stomp/stompjs	STOMP over WebSocket with auto-reconnect and heartbeats.
API docs	springdoc OpenAPI (Swagger UI)	Live, clickable documentation of every endpoint.
Testing	JUnit 5, Mockito, Spring Boot Test	28 automated tests (unit, concurrency, integration).
Build & deploy	Maven, **Docker**, Render, Vercel, MongoDB Atlas, Upstash	Container image for the backend; free cloud hosting for all parts.
Dev tools	VS Code / IntelliJ, MongoDB Compass, Postman, Git & GitHub	Coding, inspecting collections, testing APIs, version control.

4.2 Database design (MongoDB: 10 collections)

Collection	Stores	Important index
users	Account, display name, BCrypt hash, last seen	unique username, unique email
servers	Communities (name, owner, discoverable)	discoverable
server_members	User ↔ server with role OWNER / ADMIN / MEMBER	unique {serverId, userId}; userId
rooms	DIRECT, GROUP and CHANNEL conversations + last message	unique directKey; serverId
room_members	User ↔ room, role, **lastReadAt** (read receipts)	unique {roomId, userId}; userId
messages	Every message, reactions, mentions, reply, flags	**{roomId: 1, createdAt: -1}**; unique partial {roomId, senderId, clientMessageId}
invites	Invite codes, expiry, max uses, use count	unique code
friendships	Requests and friends (PENDING / ACCEPTED)	unique pairKey
notifications	Friend request, mention, added to group, joined server	{userId, createdAt}
audit_logs	Admin actions inside servers	{serverId, createdAt}

Why separate collections and not one big document? A MongoDB document is limited to 16 MB and a busy chat would exceed it. Keeping messages in their own collection with the index `{roomId: 1, createdAt: -1}` lets MongoDB load "the newest 30 messages of this room" in one index walk, no matter how many million messages exist.

4.3 Redis keys

Key	Type	Purpose
`presence:connections`	Hash	userId → number of open sockets; online when 0 → 1, offline when back to 0.
`unread:<userId>`	Hash	roomId → unread count (HINCRBY on each new message, HDEL when the chat is opened).
`rate:<action>:<user>:<window>`	String + TTL	Fixed-window rate limit: 8 messages / 5 s, 10 logins / minute.

`auth:blacklist:<tokenId>`	String + TTL	Logged-out JWTs are rejected until they would expire anyway.

4.4 Techniques

Technique	How it is applied in Nexus
Layered architecture	Controller (HTTP / WebSocket) → Service (business rules) → Repository (database). DTOs are used for every request and response, so entities are never exposed.
Polyglot persistence	Two NoSQL databases, each for what it does best: MongoDB for durable documents, Redis for hot counters.
Indexing & pagination	Compound and unique indexes; cursor pagination with `createdAt < before` instead of slow skip/offset.
Denormalisation	`rooms.lastMessage` stores a preview so the chat list is one query, not one per room.
Idempotency	The client generates a `clientMessageId` once and reuses it on retry; a unique partial index guarantees one stored message (10 parallel retries → 1 document, proven by a test).
Optimistic UI	The message appears immediately with a clock icon and is swapped with the saved copy when the server confirms.
Publish / subscribe	STOMP topics: `/topic/rooms/{id}` for a room, `/user/queue/events` private per user, `/topic/presence`.
Stateless authentication	JWT signed with HMAC-SHA; checked on every REST call (filter) and on the WebSocket CONNECT frame (interceptor).
RBAC	Roles live in `server_members`; `AccessGuard` checks Owner > Admin > Member on the backend, so hiding a button is not the only protection.
Graceful degradation	`RedisSafe` falls back to memory / MongoDB if Redis is down; MongoDB outages return 503 "retry later".
Centralised error handling	Custom exception hierarchy + `ErrorCode` enum → one JSON envelope with `errorCode` and `requestId`; the frontend maps codes to friendly messages.
Observability	Every request gets a `requestId` written in each log line and returned to the client.
Automated testing	28 tests including concurrency tests that found and fixed two real race-condition bugs.

Containerisation & CI-free deploy	Multi-stage Dockerfile (Maven build → small JRE image, non-root user) deployed on Render; frontend on Vercel.

5. CODE

The full project has about 120 source files. The most important parts are listed below (taken directly from the project source). Backend package: `com.example.crm`.

5.1 Project structure

```
Chat-Application/
├── ChatAppBackend/                 Spring Boot backend
│   └── src/main/java/com/example/crm/
│       ├── controler/              REST + WebSocket controllers
│       ├── service/ , service/impl/ business logic
│       ├── repository/             Spring Data MongoDB repositories
│       ├── entity/                 MongoDB documents (10 collections)
│       ├── dto/ (request, response, event)
│       ├── security/               JWT, filters, AccessGuard (RBAC)
│       ├── config/                 Security, WebSocket, seed data
│       ├── exception/ , enums/ErrorCode.java
│       └── util/RedisSafe.java
└── chat-frontend/                  Next.js frontend
    ├── app/                        pages (login, register, chats, servers...)
    ├── component/                  chat UI, 3D scenes, navigation
    ├── context/                    Auth, Socket, AppState, Server
    ├── services/                   API calls
    └── lib/                        ApiError, errorHandler, utils
```

5.2 MongoDB document with indexes

entity/Message.java

```
// Messages Live In Their Own Collection (Not Inside Room) -> No 16MB Limit, Real Pagination
@Document(collection = "messages")
@CompoundIndex(name = "room_created", def = "{'roomId': 1, 'createdAt': -1}")
// Idempotency Key: A Client Retry With The Same clientMessageId Hits This Unique Index Instead Of
// Creating A Second Message. Partial Filter -> System Messages (No clientMessageId) Are Not Indexed.
@CompoundIndex(name = "idempotency_key", def = "{'roomId': 1, 'senderId': 1, 'clientMessageId': 1}",
unique = true,
        partialFilter = "{ 'clientMessageId': { '$type': 'string' } }")
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Message {
    @Id
    private String id;
    private String roomId;
    private String senderId;
    // Generated By The Client Once Per Message And Reused On Every Retry Of That Message
    private String clientMessageId;
    private String content;
    private MessageType type;
    private String replyToId;
    @Builder.Default
    private Set<String> mentions = new LinkedHashSet<>();
```

```
    // Emoji -> UserIds Who Reacted
    @Builder.Default
    private Map<String, Set<String>> reactions = new LinkedHashMap<>();
    private boolean edited;
    private boolean deleted;
    private boolean pinned;
    private Instant createdAt;
    private Instant editedAt;
}
```

5.3 Repository (queries generated by Spring Data from method names)

repository/MessageRepository.java

```
@Repository
public interface MessageRepository extends MongoRepository<Message, String> {

    Optional<Message> findByRoomIdAndSenderIdAndClientMessageId(String roomId, String senderId,
String clientMessageId);

    Slice<Message> findByRoomIdOrderByCreatedAtDesc(String roomId, Pageable pageable);

    Slice<Message> findByRoomIdAndCreatedAtBeforeOrderByCreatedAtDesc(String roomId, Instant before,
Pageable pageable);

    List<Message> findByRoomIdAndPinnedTrueOrderByCreatedAtDesc(String roomId);

    List<Message>
findTop50ByRoomIdAndDeletedFalseAndContentContainingIgnoreCaseOrderByCreatedAtDesc(String roomId,
String content);

    long countByRoomIdAndCreatedAtAfterAndSenderIdNot(String roomId, Instant after, String senderId);

    long countByRoomIdAndSenderIdNot(String roomId, String senderId);

    void deleteByRoomId(String roomId);

    void deleteByRoomIdIn(java.util.Collection<String> roomIds);
}
```

5.4 Sending a message (service layer)

Steps: check membership → idempotency check → rate limit → save in MongoDB → update room preview → broadcast over WebSocket → increase unread counters in Redis → notify mentioned users.

service/impl/ChatServiceImpl.java (sendMessage)

```
public class ChatServiceImpl implements ChatService {
    private static final Pattern MENTION = Pattern.compile("@([a-zA-Z0-9_.]{3,20})");
    private static final int PREVIEW_LENGTH = 80;
    private static final String MESSAGE_RATE_ACTION = "message";
    private static final int MESSAGE_RATE_LIMIT = 8;
    private static final int MESSAGE_RATE_WINDOW_SECONDS = 5;

    private final MessageRepository messageRepository;
    private final RoomRepository roomRepository;
    private final RoomMemberRepository roomMemberRepository;
```

```
    private final UserRepository userRepository;
    private final AccessGuard accessGuard;
    private final MessageMapper messageMapper;
    private final RealtimeService realtimeService;
    private final UnreadService unreadService;
    private final RateLimitService rateLimitService;
    private final NotificationService notificationService;
    private final AuditLogService auditLogService;

    @Override
    public MessageResponse sendMessage(String roomId, SendMessageRequest request, String userId) {
        Room room = accessGuard.requireRoom(roomId);
        RoomMember senderMembership = accessGuard.requireRoomMember(roomId, userId);

        // Idempotency: A Network Retry Of A Message We Already Stored Returns The Stored One.
        // Checked Before Rate Limiting So Retries Do Not Burn The User's Quota.
        Message alreadySaved = findByIdempotencyKey(roomId, userId, request.getClientMessageId());
        if (alreadySaved != null) {
            return redeliver(alreadySaved);
        }
        // Max 8 Messages In 5 Seconds (Redis Counter) -> RateLimitException (429).
        // Soft Limit By Design: We Count Only AFTER A Successful Save, So Concurrent Retries Of One
Message
        // Never Burn Quota. The Price: A Burst Of Distinct Concurrent Messages May Overshoot
Slightly.
        rateLimitService.ensureBelowLimit(userId, MESSAGE_RATE_ACTION, MESSAGE_RATE_LIMIT,
MESSAGE_RATE_WINDOW_SECONDS);
        String content = request.getContent().trim();
        if (content.isEmpty()) {
            throw new BadRequestException("Message cannot be empty", ErrorCode.VALIDATION_FAILED);
        }
        if (request.getReplyToId() != null) {
            messageRepository.findById(request.getReplyToId())
                    .filter(m -> m.getRoomId().equals(roomId))
                    .orElseThrow(() -> new BadRequestException("You can only reply to a message in
this room", ErrorCode.BAD_REQUEST));
        }
        List<RoomMember> members = roomMemberRepository.findByRoomId(roomId);
        Set<String> mentions = resolveMentions(content, members);
        Instant now = Instant.now();

        Message message;
        try {
            message = messageRepository.save(Message.builder()
                    .roomId(roomId).senderId(userId).clientMessageId(request.getClientMessageId())
                    .content(content).type(MessageType.TEXT)
                    .replyToId(request.getReplyToId()).mentions(mentions).createdAt(now).build());
        } catch (DuplicateKeyException e) {
            // Two Copies Of The Same Message Raced Past The Check Above (e.g. WebSocket Send + REST
Retry).
            // The Unique Index Let Exactly One Win; The Loser Answers With The Winner's Message.
            return redeliver(findByIdempotencyKey(roomId, userId, request.getClientMessageId()));
        }
        rateLimitService.record(userId, MESSAGE_RATE_ACTION, MESSAGE_RATE_WINDOW_SECONDS);

        User sender = userRepository.findById(userId).orElseThrow();
        room.setLastMessage(new Room.LastMessage(message.getId(), userId, sender.getDisplayName(),
preview(content), now));
        room.setLastActivityAt(now);
        roomRepository.save(room);

        // Sender Has Obviously Read Everything Up To Now
        senderMembership.setLastReadAt(now);
        roomMemberRepository.save(senderMembership);
```

```
        MessageResponse response = messageMapper.toResponse(message);
        realtimeService.toRoom(roomId, ChatEvent.of(ChatEventType.MESSAGE_CREATED, roomId,
response));

        // Unread Badges For Everyone Else (Redis HINCRBY) Pushed To Their Private Queue
        for (RoomMember member : members) {
            if (!member.getUserId().equals(userId)) {
                long count = unreadService.increment(member);
                realtimeService.toUser(member.getUserId(), ChatEvent.of(ChatEventType.UNREAD, roomId,
                        new UnreadEvent(roomId, room.getServerId(), count)));
            }
        }
        for (String mentioned : mentions) {
            if (!mentioned.equals(userId)) {
                notificationService.notify(mentioned, NotificationType.MENTION,
                        sender.getDisplayName() + " mentioned you" + (room.getName() != null ? " in
#" + room.getName() : ""),
                        preview(content), link(room), userId);
            }
        }
```

5.5 Redis: unread counters with MongoDB fallback

service/impl/UnreadServiceImpl.java

```
/**
 * Unread badges. Redis hash "unread:{userId}" holds roomId -> count, so the sidebar
 * never has to count messages in MongoDB. If Redis is down we count from MongoDB using lastReadAt.
 */
@Service
@RequiredArgsConstructor
public class UnreadServiceImpl implements UnreadService {
    private static final String PREFIX = "unread:";

    private final StringRedisTemplate redis;
    private final RedisSafe redisSafe;
    private final MessageRepository messageRepository;

    @Override
    public long increment(RoomMember member) {
        return redisSafe.call(
                () -> redis.opsForHash().increment(PREFIX + member.getUserId(), member.getRoomId(),
1),
                () -> countFromMongo(member));
    }

    @Override
    public void reset(String userId, String roomId) {
        redisSafe.run(() -> redis.opsForHash().delete(PREFIX + userId, roomId), () -> {});
    }

    @Override
    public Map<String, Long> counts(String userId, Collection<RoomMember> memberships) {
        Map<String, Long> result = new HashMap<>();
        if (memberships.isEmpty()) {
            return result;
        }
        List<Object> roomIds = new ArrayList<>();
        memberships.forEach(m -> roomIds.add(m.getRoomId()));
        return redisSafe.call(() -> {
```

```
            List<Object> values = redis.opsForHash().multiGet(PREFIX + userId, roomIds);
            for (int i = 0; i < roomIds.size(); i++) {
                Object v = values.get(i);
                result.put(roomIds.get(i).toString(), v == null ? 0L : Long.parseLong(v.toString()));
            }
            return result;
        }, () -> {
            memberships.forEach(m -> result.put(m.getRoomId(), countFromMongo(m)));
            return result;
        });
    }

    private long countFromMongo(RoomMember member) {
        if (member.getLastReadAt() == null) {
            return messageRepository.countByRoomIdAndSenderIdNot(member.getRoomId(),
member.getUserId());
        }
        return messageRepository.countByRoomIdAndCreatedAtAfterAndSenderIdNot(
                member.getRoomId(), member.getLastReadAt(), member.getUserId());
    }
}
```

5.6 Redis: rate limiting

service/impl/RateLimitServiceImpl.java

```
    private final RedisSafe redisSafe;

    // Atomic INCR: Even Under Concurrency Exactly "limit" Calls Per Window Get Through (Used For
Login)
    @Override
    public void check(String userId, String action, int limit, int windowSeconds) {
        failIfOver(increment(key(userId, action, windowSeconds), windowSeconds), limit,
windowSeconds);
    }

    @Override
    public void ensureBelowLimit(String userId, String action, int limit, int windowSeconds) {
        String key = key(userId, action, windowSeconds);
        // "Count >= limit" Because This Call Itself Is Not Counted Yet
        long count = redisSafe.call(() -> {
            String value = redis.opsForValue().get(key);
            return value == null ? 0L : Long.parseLong(value);
        }, () -> 0L);
        failIfOver(count + 1, limit, windowSeconds);
    }

    @Override
    public void record(String userId, String action, int windowSeconds) {
        increment(key(userId, action, windowSeconds), windowSeconds);
    }

    private long increment(String key, int windowSeconds) {
        // If Redis Is Down We Let The Request Through (Fail Open): Losing Spam Protection For A
While
        // Is Better Than Blocking Every User's Chat
        return redisSafe.call(() -> {
            Long value = redis.opsForValue().increment(key);
            if (value != null && value == 1) {
                redis.expire(key, Duration.ofSeconds(windowSeconds));
            }
```

```
            return value == null ? 0L : value;
        }, () -> 0L);
    }

    private static String key(String userId, String action, int windowSeconds) {
        long window = System.currentTimeMillis() / 1000 / windowSeconds;
        return "rate:" + action + ":" + userId + ":" + window;
    }

    private static void failIfOver(long count, int limit, int windowSeconds) {
        if (count > limit) {
            throw new RateLimitException("Slow down! Max " + limit + " per " + windowSeconds + "s",
ErrorCode.RATE_LIMITED);
        }
    }
}
```

5.7 Redis safety wrapper

util/RedisSafe.java

```
/**
 * Runs a Redis call and falls back when Redis is down, so the chat keeps working
 * (with in-memory / MongoDB fallbacks) during a demo even if Redis is not started.
 * After a failure Redis is skipped for 30 seconds instead of waiting for a timeout on every call.
 */
@Slf4j
@Component
public class RedisSafe {
    private static final long RETRY_AFTER_MS = 30_000;
    private final AtomicLong downSince = new AtomicLong(0);

    public <T> T call(Supplier<T> redisCall, Supplier<T> fallback) {
        long down = downSince.get();
        if (down != 0 && System.currentTimeMillis() - down < RETRY_AFTER_MS) {
            return fallback.get();
        }
        try {
            T result = redisCall.get();
            if (down != 0) {
                downSince.set(0);
                log.info("Redis is reachable again");
            }
            return result;
        } catch (RuntimeException e) {
            if (downSince.getAndSet(System.currentTimeMillis()) == 0) {
                log.warn("Redis unavailable, using fallback: {}", e.getMessage());
            }
            return fallback.get();
        }
    }

    public void run(Runnable redisCall, Runnable fallback) {
        call(() -> {
            redisCall.run();
            return null;
        }, () -> {
            fallback.run();
            return null;
        });
    }
```

```
    public boolean isUp() {
        return downSince.get() == 0;
    }
}
```

5.8 Role-based access control

security/AccessGuard.java

```
/**
 * Central place for "is this user allowed?" checks. Every service calls these,
 * so permission rules live in one file instead of being copied everywhere.
 */
@Component
@RequiredArgsConstructor
public class AccessGuard {
    private final RoomRepository roomRepository;
    private final RoomMemberRepository roomMemberRepository;
    private final ServerRepository serverRepository;
    private final ServerMemberRepository serverMemberRepository;

    public Room requireRoom(String roomId) {
        return roomRepository.findById(roomId)
                .orElseThrow(() -> new ResourceNotFoundException("Room not found for id " + roomId,
ErrorCode.ROOM_NOT_FOUND));
    }

    public RoomMember requireRoomMember(String roomId, String userId) {
        return roomMemberRepository.findByRoomIdAndUserId(roomId, userId)
                .orElseThrow(() -> new ForbiddenException(ErrorCode.NOT_ROOM_MEMBER));
    }

    public boolean isRoomMember(String roomId, String userId) {
        return roomMemberRepository.existsByRoomIdAndUserId(roomId, userId);
    }

    public Server requireServer(String serverId) {
        return serverRepository.findById(serverId)
                .orElseThrow(() -> new ResourceNotFoundException("Server not found for id " +
serverId, ErrorCode.SERVER_NOT_FOUND));
    }

    public ServerMember requireServerMember(String serverId, String userId) {
        return serverMemberRepository.findByServerIdAndUserId(serverId, userId)
                .orElseThrow(() -> new ForbiddenException(ErrorCode.NOT_SERVER_MEMBER));
    }

    public ServerMember requireServerRole(String serverId, String userId, MemberRole minimum) {
        ServerMember member = requireServerMember(serverId, userId);
        if (!member.getRole().atLeast(minimum)) {
            throw new ForbiddenException("Only " + minimum.name().toLowerCase() + "s can do this",
ErrorCode.ACCESS_DENIED);
        }
        return member;
    }

    /**
     * Moderation rights inside a room: server ADMIN+ for channels, group ADMIN+ for groups.
     */
    public boolean canModerate(Room room, String userId) {
```

```
        if (room.getServerId() != null) {
            return serverMemberRepository.findByServerIdAndUserId(room.getServerId(), userId)
                    .map(m -> m.getRole().atLeast(MemberRole.ADMIN)).orElse(false);
        }
        return roomMemberRepository.findByRoomIdAndUserId(room.getId(), userId)
                .map(m -> m.getRole() != null &&
m.getRole().atLeast(MemberRole.ADMIN)).orElse(false);
    }
}
```

5.9 JWT token service

security/JwtService.java

```
@Service
public class JwtService {
    private final SecretKey key;
    private final Duration expiration;

    public JwtService(@Value("${app.jwt.secret}") String secret,
            @Value("${app.jwt.expiration-hours:72}") long expirationHours) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expiration = Duration.ofHours(expirationHours);
    }

    public String generateToken(User user) {
        Instant now = Instant.now();
        return Jwts.builder()
                .id(UUID.randomUUID().toString())
                .subject(user.getId())
                .claim("username", user.getUsername())
                .issuedAt(Date.from(now))
                .expiration(Date.from(now.plus(expiration)))
                .signWith(key)
                .compact();
    }

    public Instant expiresAt(String token) {
        return parse(token).getExpiration().toInstant();
    }

    /**
     * @throws UnauthorizedException with TOKEN_EXPIRED or UNAUTHORIZED when the token is not usable
     */
    public Claims parse(String token) {
        try {
            return Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload();
        } catch (ExpiredJwtException e) {
            throw new UnauthorizedException(ErrorCode.TOKEN_EXPIRED);
        } catch (JwtException | IllegalArgumentException e) {
            throw new UnauthorizedException("Invalid token", ErrorCode.UNAUTHORIZED);
        }
    }

    public AuthUser toAuthUser(Claims claims) {
        return new AuthUser(claims.getSubject(), claims.get("username", String.class));
    }
}
```

5.10 WebSocket configuration

config/WebSocketConfig.java

```
@Configuration
@EnableWebSocketMessageBroker
@RequiredArgsConstructor
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    private final WebSocketAuthInterceptor webSocketAuthInterceptor;

    @Value("${app.cors.allowed-origins}")
    private List<String> allowedOrigins;

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        ThreadPoolTaskScheduler heartbeatScheduler = new ThreadPoolTaskScheduler();
        heartbeatScheduler.setPoolSize(1);
        heartbeatScheduler.setThreadNamePrefix("ws-heartbeat-");
        heartbeatScheduler.initialize();

        // /topic/rooms/{roomId} -> Everyone In A Room, /topic/presence -> Online/Offline
        // /user/queue/...       -> Private To One User (Notifications, Unread Counts, Errors)
        config.enableSimpleBroker("/topic", "/queue")
                .setHeartbeatValue(new long[] { 10000, 10000 })
                .setTaskScheduler(heartbeatScheduler);
        config.setApplicationDestinationPrefixes("/app");
        config.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        String[] origins = allowedOrigins.toArray(String[]::new);
        // Plain WebSocket (Used By The Next.js Client)
        registry.addEndpoint("/ws").setAllowedOriginPatterns(origins);
        // SockJS Fall Back For Old Browsers
        registry.addEndpoint("/chat").setAllowedOriginPatterns(origins).withSockJS();
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(webSocketAuthInterceptor);
    }
}
```

5.11 Chat controller (WebSocket + REST)

controler/ChatController.java

```
@Slf4j
@RestController
@RequestMapping("/api/v1/messages")
@RequiredArgsConstructor
public class ChatController {
    private final ChatService chatService;

    // ---------------- WebSocket (STOMP) ----------------

    // Client Sends To /app/rooms/{roomId}/send -> Saved -> Broadcast On /topic/rooms/{roomId}
    @MessageMapping("/rooms/{roomId}/send")
    public void sendMessage(@DestinationVariable String roomId, @Valid @Payload SendMessageRequest
```

```
request, Principal principal) {
        chatService.sendMessage(roomId, request, principal.getName());
    }

    // Client Sends To /app/rooms/{roomId}/typing -> "Razi is typing..." On /topic/rooms/{roomId}
    @MessageMapping("/rooms/{roomId}/typing")
    public void typing(@DestinationVariable String roomId, @Payload TypingPayload payload, Principal
principal) {
        chatService.typing(roomId, payload.isTyping(), principal.getName());
    }

    // WebSocket Errors Use The Same Response Envelope, Delivered Privately On /user/queue/errors
    @MessageExceptionHandler(ChatAppException.class)
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public Response<Void> handleChatAppException(ChatAppException ex) {
        return Response.error(ex.getErrorCode(), ex.getMessage(), ex.getMeta());
    }

    @MessageExceptionHandler(MethodArgumentNotValidException.class)
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public Response<Void> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        if (ex.getBindingResult() != null) {
            ex.getBindingResult().getFieldErrors().forEach(fe ->
fieldErrors.putIfAbsent(fe.getField(), fe.getDefaultMessage()));
        }
        return Response.error(ErrorCode.VALIDATION_FAILED, "Message is not valid", fieldErrors);
    }

    @MessageExceptionHandler(Exception.class)
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public Response<Void> handleUnexpected(Exception ex) {
        log.error("WebSocket error", ex);
        return Response.error(ErrorCode.INTERNAL_ERROR, ErrorCode.INTERNAL_ERROR.getDefaultMessage(),
null);
    }

    // ---------------- REST (Edit / Delete / React / Pin) ----------------

    @PutMapping("/{messageId}")
    public ResponseEntity<Response<MessageResponse>> edit(@AuthenticationPrincipal AuthUser me,
@PathVariable String messageId,
            @Valid @RequestBody EditMessageRequest request) {
        return ResponseEntity.ok(Response.ok(chatService.edit(messageId, request.getContent(),
me.id()), "Message Edited"));
    }

    @DeleteMapping("/{messageId}")
    public ResponseEntity<Response<MessageResponse>> delete(@AuthenticationPrincipal AuthUser me,
@PathVariable String messageId) {
        return ResponseEntity.ok(Response.ok(chatService.delete(messageId, me.id()), "Message
Deleted"));
    }

    @PostMapping("/{messageId}/reactions")
    public ResponseEntity<Response<MessageResponse>> react(@AuthenticationPrincipal AuthUser me,
@PathVariable String messageId,
            @Valid @RequestBody ReactionRequest request) {
        return ResponseEntity.ok(Response.ok(chatService.react(messageId, request.getEmoji(),
me.id()), "Reaction Updated"));
    }

    @PostMapping("/{messageId}/pin")
    public ResponseEntity<Response<MessageResponse>> pin(@AuthenticationPrincipal AuthUser me,
```

```
@PathVariable String messageId) {
        return ResponseEntity.ok(Response.ok(chatService.togglePin(messageId, me.id()), "Pin
Updated"));
    }
}
```

5.12 Global exception handling

exception/ChatAppException.java

```
/**
 * Base class of all our own exceptions. RestResponseExceptionHandler turns any of these
 * into a Response with the matching HTTP status and error code.
 */
public class ChatAppException extends RuntimeException {
    private final ErrorCode errorCode;
    private final transient Object meta;

    public ChatAppException(ErrorCode errorCode) {
        this(errorCode, errorCode.getDefaultMessage(), null);
    }

    public ChatAppException(ErrorCode errorCode, String message) {
        this(errorCode, message, null);
    }

    public ChatAppException(ErrorCode errorCode, String message, Object meta) {
        super(message);
        this.errorCode = errorCode;
        this.meta = meta;
    }

    public ErrorCode getErrorCode() {
        return errorCode;
    }

    public Object getMeta() {
        return meta;
    }
}
```

exception/RestResponseExceptionHandler.java

```
@ControllerAdvice
@Slf4j
@Order(Ordered.HIGHEST_PRECEDENCE)
public class RestResponseExceptionHandler {

    // All Our Own Exceptions (NotFound, Duplicate, Forbidden, RateLimit ...)
    @ExceptionHandler(ChatAppException.class)
    public ResponseEntity<Response<Void>> handleChatAppException(ChatAppException ex) {
        log.debug("Handled {} -> {}", ex.getErrorCode(), ex.getMessage());
        return build(ex.getErrorCode(), ex.getMessage(), ex.getMeta());
    }

    // @Valid Failed -> Return Every Field Error In errorMeta So The Form Can Show It Under The Field
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Response<Void>> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
```

```
                .forEach(fe -> fieldErrors.putIfAbsent(fe.getField(), fe.getDefaultMessage()));
        return build(ErrorCode.VALIDATION_FAILED, "Please fix the highlighted fields", fieldErrors);
    }

    @ExceptionHandler({ HttpMessageNotReadableException.class,
MethodArgumentTypeMismatchException.class })
    public ResponseEntity<Response<Void>> handleBadBody(Exception ex) {
        return build(ErrorCode.BAD_REQUEST, "Malformed request", null);
    }

    @ExceptionHandler(HttpRequestMethodNotSupportedException.class)
    public ResponseEntity<Response<Void>> handleMethod(HttpRequestMethodNotSupportedException ex) {
        return build(ErrorCode.BAD_REQUEST, ex.getMessage(), null);
    }

    @ExceptionHandler(NoResourceFoundException.class)
    public ResponseEntity<Response<Void>> handleNoResource(NoResourceFoundException ex) {
        Response<Void> body = Response.error(ErrorCode.BAD_REQUEST, "Endpoint not found", null);
        body.setErrorCode("REQ_404");
        return ResponseEntity.status(404).body(body);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<Response<Void>> handleAccessDenied(AccessDeniedException ex) {
        return build(ErrorCode.ACCESS_DENIED, ErrorCode.ACCESS_DENIED.getDefaultMessage(), null);
    }

    // Unique Index Violation That No Service Translated Itself (e.g. Two Identical Requests Racing)
    @ExceptionHandler(DuplicateKeyException.class)
    public ResponseEntity<Response<Void>> handleDuplicateKey(DuplicateKeyException ex) {
        log.warn("Duplicate key: {}", ex.getMostSpecificCause().getMessage());
        return build(ErrorCode.CONFLICT, ErrorCode.CONFLICT.getDefaultMessage(), null);
    }

    // MongoDB Down / Timed Out -> 503 Tells The Client "Retry Later", Unlike A 500 Which Means "Bug"
    @ExceptionHandler({ DataAccessResourceFailureException.class, TransientDataAccessException.class
})
    public ResponseEntity<Response<Void>> handleDatabaseUnavailable(DataAccessException ex) {
        log.error("Database unavailable: {}", ex.getMostSpecificCause().getMessage());
        return build(ErrorCode.DATABASE_UNAVAILABLE,
ErrorCode.DATABASE_UNAVAILABLE.getDefaultMessage(), null);
    }

    // Anything We Did Not Expect -> 500 Without Leaking Internals
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Response<Void>> handleUnexpected(Exception ex) {
        log.error("Unexpected error", ex);
        return build(ErrorCode.INTERNAL_ERROR, ErrorCode.INTERNAL_ERROR.getDefaultMessage(), null);
    }

    private ResponseEntity<Response<Void>> build(ErrorCode code, String message, Object meta) {
        return ResponseEntity.status(code.getStatus()).body(Response.error(code, message, meta));
    }
}
```

5.13 Frontend: WebSocket connection

chat-frontend/context/SocketContext.tsx (connection part)

```
  useEffect(() => {
    if (!token) return;
```

```
    const emit = (event: ChatEvent) => listeners.current.forEach((l) => l(event));
    const client = new Client({
      brokerURL: WsUrl,
      connectHeaders: { Authorization: `Bearer ${token}` },
      reconnectDelay: 3000,
      heartbeatIncoming: 10000,
      heartbeatOutgoing: 10000,
      debug: () => {},
    });

    client.onConnect = () => {
      setConnected(true);
      client.subscribe("/user/queue/events", (m) => emit(JSON.parse(m.body)));
      client.subscribe("/topic/presence", (m) => emit(JSON.parse(m.body)));
      // Backend @MessageExceptionHandler -> Same Response Envelope As REST -> Same Error Handler
      client.subscribe("/user/queue/errors", (m) => {
        const body = JSON.parse(m.body) as ApiResponse<unknown>;
        handleError(ApiError.fromResponse(body, 400));
      });
      registry.current.forEach((reg) => attach(client, reg));
    };

    // ERROR Frame From WebSocketAuthInterceptor, e.g. "AUTH_403: Session expired"
    client.onStompError = (frame) => {
      const text = frame.headers["message"] ?? "";
      if (text.startsWith("AUTH_")) {
        client.deactivate();
        window.dispatchEvent(new CustomEvent(AUTH_EXPIRED_EVENT));
      } else if (text) {
        handleError(new ApiError(text.replace(/^\w+: /, ""), text.split(":")[0] ||
ErrorCodes.UNKNOWN));
      }
    };
    client.onWebSocketClose = () => setConnected(false);

    client.activate();
    clientRef.current = client;
    const regs = registry.current;
    return () => {
      regs.forEach((reg) => (reg.stomp = undefined));
      client.deactivate();
      clientRef.current = null;
      setConnected(false);
    };
  }, [token, attach]);
```

5.14 Frontend: one error type for the whole app

chat-frontend/lib/ApiError.ts

```
import type { ApiResponse } from "@/types";

/**
 * The ONE error type the whole frontend deals with.
 * Every failure (HTTP error, network down, timeout, WebSocket error frame) becomes an ApiError,
 * carrying the same errorCode the backend's ErrorCode enum sends.
 */
export class ApiError extends Error {
  readonly code: string;
  readonly status: number;
  readonly meta?: Record<string, string>;
```

```
  // Backend Correlation Id: Matches The Server Log Lines Of The Failed Request
  readonly requestId?: string;

  constructor(message: string, code: string, status = 0, meta?: Record<string, string>, requestId?:
string) {
    super(message);
    this.name = "ApiError";
    this.code = code;
    this.status = status;
    this.meta = meta;
    this.requestId = requestId;
  }

  static fromResponse(body: Partial<ApiResponse<unknown>> | undefined, status: number): ApiError {
    const meta =
      body?.errorMeta && typeof body.errorMeta === "object" ? (body.errorMeta as Record<string,
string>) : undefined;
    return new ApiError(body?.message || "Something went wrong", body?.errorCode || `HTTP_${status}`,
status, meta, body?.requestId);
  }

  static network(): ApiError {
    return new ApiError("Cannot reach the server. Is the backend running?", ErrorCodes.NETWORK_ERROR,
0);
  }

  static timeout(): ApiError {
    return new ApiError("The server took too long to answer", ErrorCodes.TIMEOUT, 0);
  }

  get isAuthError(): boolean {
    return this.status === 401;
  }

  get isValidation(): boolean {
    return this.code === ErrorCodes.VALIDATION_FAILED || (this.meta !== undefined &&
Object.keys(this.meta).length > 0);
  }
}

// Same Codes As ErrorCode.java + Client-Only Ones
export const ErrorCodes = {
  NETWORK_ERROR: "NET_000",
  TIMEOUT: "NET_408",
  UNKNOWN: "CLI_000",
  VALIDATION_FAILED: "VAL_400",
  BAD_REQUEST: "REQ_400",
  INTERNAL_ERROR: "SYS_500",
  RATE_LIMITED: "RATE_429",
  CONFLICT: "REQ_409",
  DATABASE_UNAVAILABLE: "DB_503",
  UNAUTHORIZED: "AUTH_401",
  INVALID_CREDENTIALS: "AUTH_402",
  TOKEN_EXPIRED: "AUTH_403",
  ACCESS_DENIED: "AUTH_404",
  USER_NOT_FOUND: "USER_404",
  USERNAME_TAKEN: "USER_409",
  EMAIL_TAKEN: "USER_410",
  ROOM_NOT_FOUND: "ROOM_404",
  NOT_ROOM_MEMBER: "ROOM_403",
  SERVER_NOT_FOUND: "SRV_404",
  NOT_SERVER_MEMBER: "SRV_403",
  INVITE_NOT_FOUND: "INV_404",
  INVITE_EXPIRED: "INV_410",
```

```
  MESSAGE_NOT_FOUND: "MSG_404",
  FRIEND_REQUEST_EXISTS: "FRD_409",
} as const;

export const isApiError = (e: unknown): e is ApiError => e instanceof ApiError;
```

chat-frontend/lib/errorHandler.ts

```
import toast from "react-hot-toast";
import { ApiError, ErrorCodes, isApiError } from "./ApiError";

// Friendly Text Per Error Code (Falls Back To The Server's Message)
const friendlyMessages: Record<string, string> = {
  [ErrorCodes.NETWORK_ERROR]: "Can't reach the server. If it was asleep (free hosting) it is waking
up now, so try again in ~30 seconds.",
  [ErrorCodes.TIMEOUT]: "The request timed out. Please try again.",
  [ErrorCodes.INTERNAL_ERROR]: "Something broke on our side. Please try again.",
  [ErrorCodes.DATABASE_UNAVAILABLE]: "Our database is temporarily unreachable. Please retry in a
moment.",
  [ErrorCodes.TOKEN_EXPIRED]: "Your session expired. Please log in again.",
  [ErrorCodes.UNAUTHORIZED]: "Please log in to continue.",
  [ErrorCodes.ACCESS_DENIED]: "You don't have permission to do that.",
  [ErrorCodes.NOT_ROOM_MEMBER]: "You are not part of this conversation.",
  [ErrorCodes.NOT_SERVER_MEMBER]: "You are not a member of this server.",
  [ErrorCodes.INVITE_EXPIRED]: "This invite link has expired.",
  [ErrorCodes.INVITE_NOT_FOUND]: "This invite link is invalid.",
};

export function toApiError(error: unknown): ApiError {
  if (isApiError(error)) return error;
  if (error instanceof Error) return new ApiError(error.message, ErrorCodes.UNKNOWN);
  return new ApiError("Unexpected error", ErrorCodes.UNKNOWN);
}

export function getErrorMessage(error: unknown): string {
  const e = toApiError(error);
  return friendlyMessages[e.code] ?? e.message;
}

interface HandleOptions {
  // Formik's setErrors -> Field Errors From errorMeta Appear Under The Matching Input
  setFieldErrors?: (errors: Record<string, string>) => void;
  // Don't Toast (e.g. When The Page Shows The Error Itself)
  silent?: boolean;
  fallback?: string;
}

/**
 * The single place UI code sends errors to.
 *  - validation errors -> under the form fields
 *  - everything else   -> a toast with a friendly message
 */
export function handleError(error: unknown, options: HandleOptions = {}): ApiError {
  const e = toApiError(error);
  if (e.meta && options.setFieldErrors) {
    options.setFieldErrors(e.meta);
  }
  if (!options.silent) {
    const base = friendlyMessages[e.code] ?? e.message ?? options.fallback;
    // Server Failures Show The Reference So A Bug Report Can Be Matched To The Backend Logs
    const text = e.status >= 500 && e.requestId ? `${base} (ref: ${e.requestId})` : base;
    toast.error(text, { id: e.code === ErrorCodes.RATE_LIMITED ? "rate" : undefined });
```

```
  }
  if (process.env.NODE_ENV !== "production") {
    console.warn(`[${e.code}] ${e.message}`, e.meta ?? "");
  }
  return e;
}
```

5. OUTPUT SCREENSHOTS

Screenshots of the running application. The backend runs on Render (Docker, Singapore region) and the frontend in the browser.

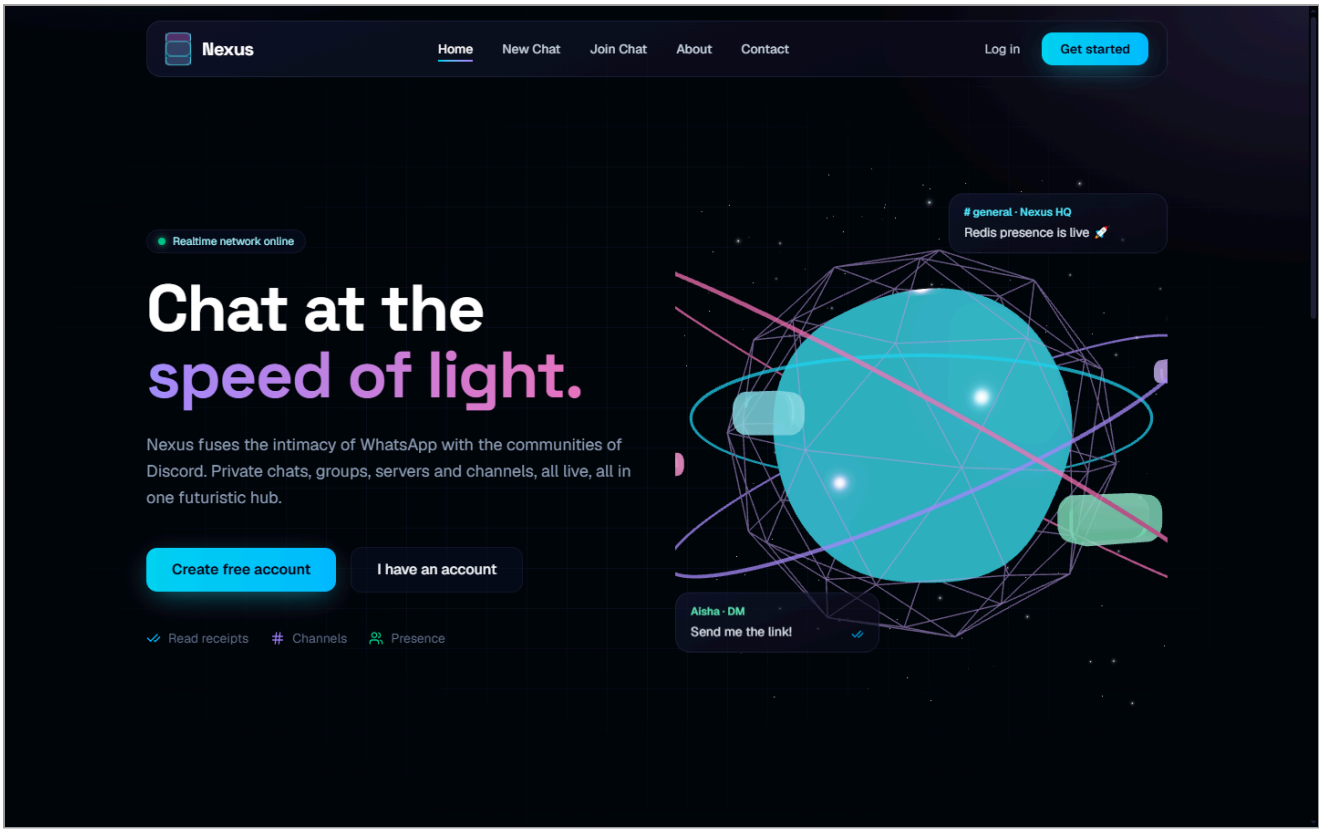


Figure 1: Landing page with the animated Three.js 3D scene

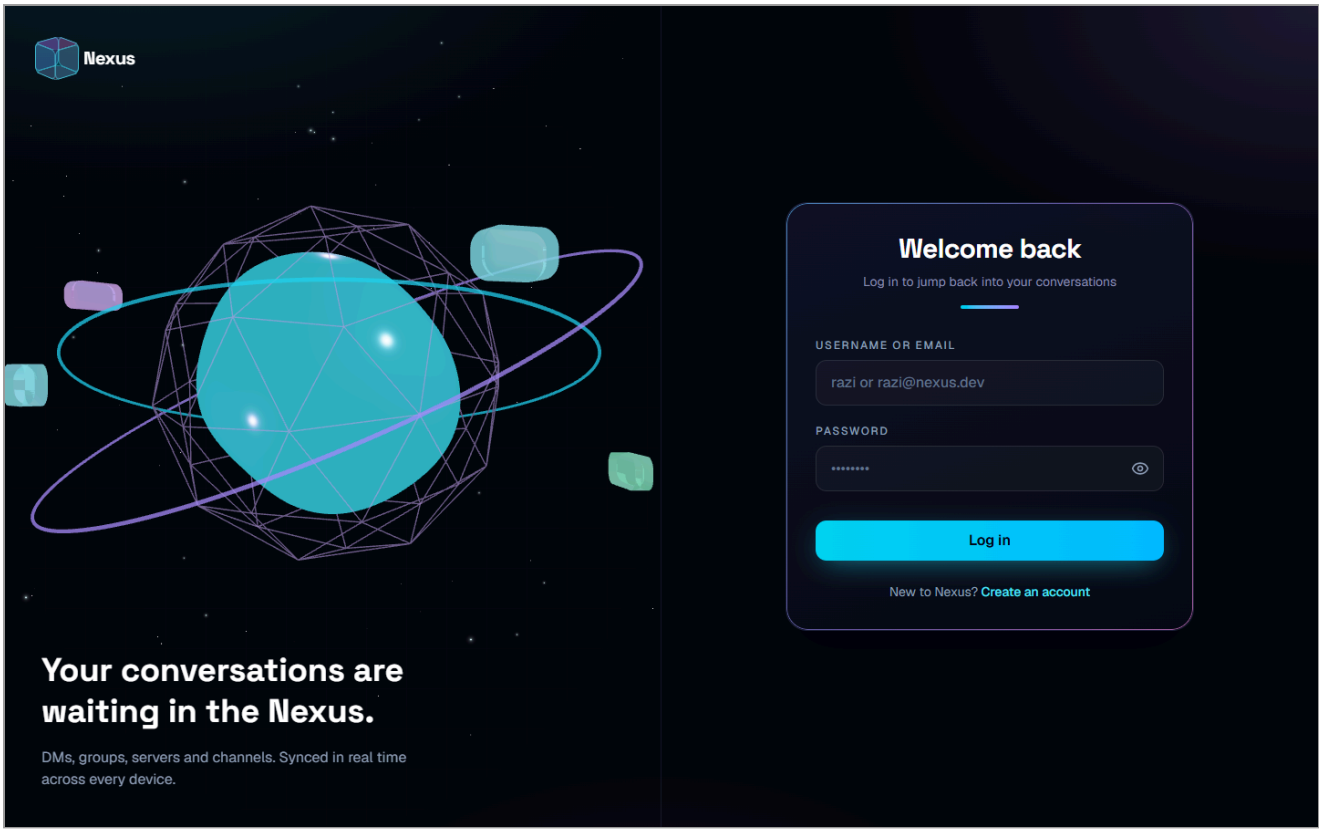


Figure 2: Login page (JWT authentication)

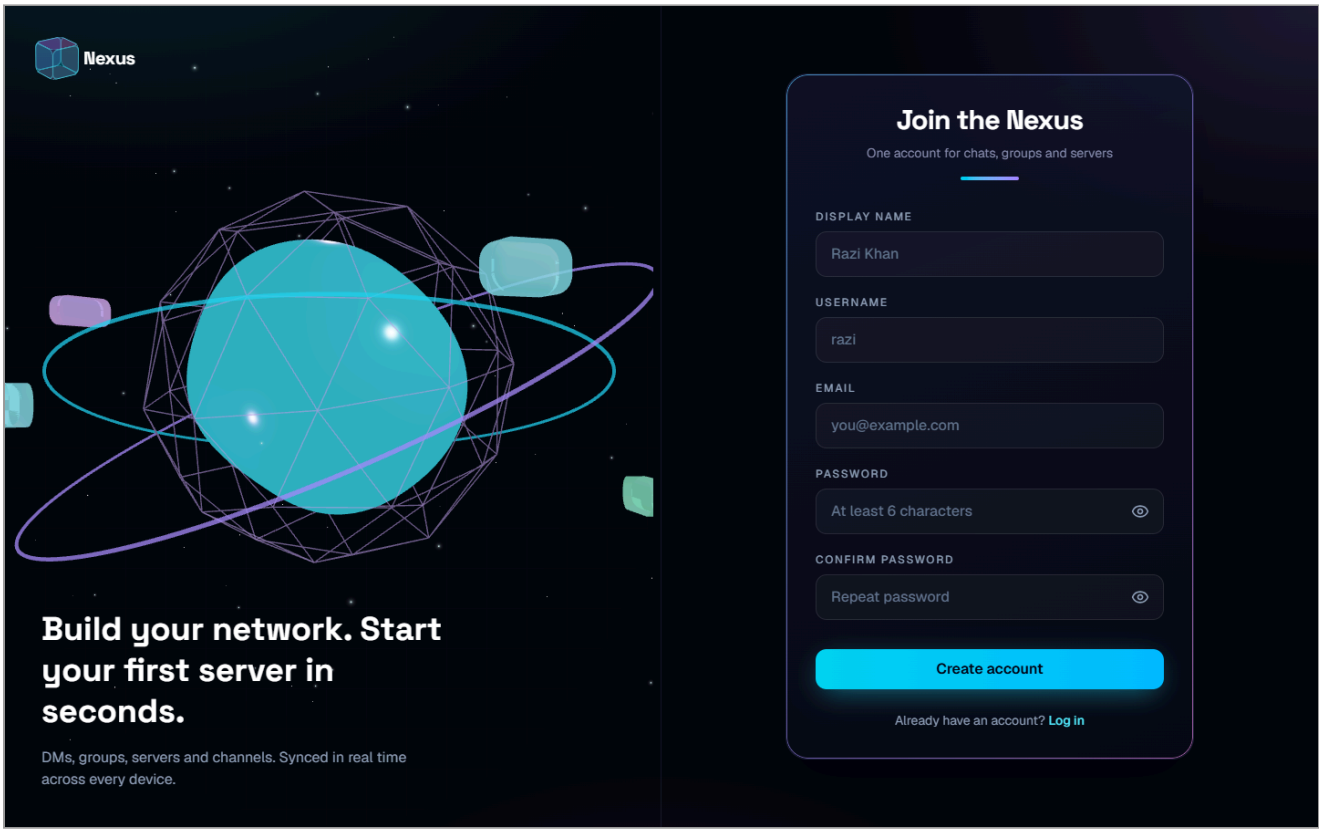


Figure 3: Registration page with field validation (Formik + Yup)



Figure 4: About page describing the frontend, backend, MongoDB collections and Redis

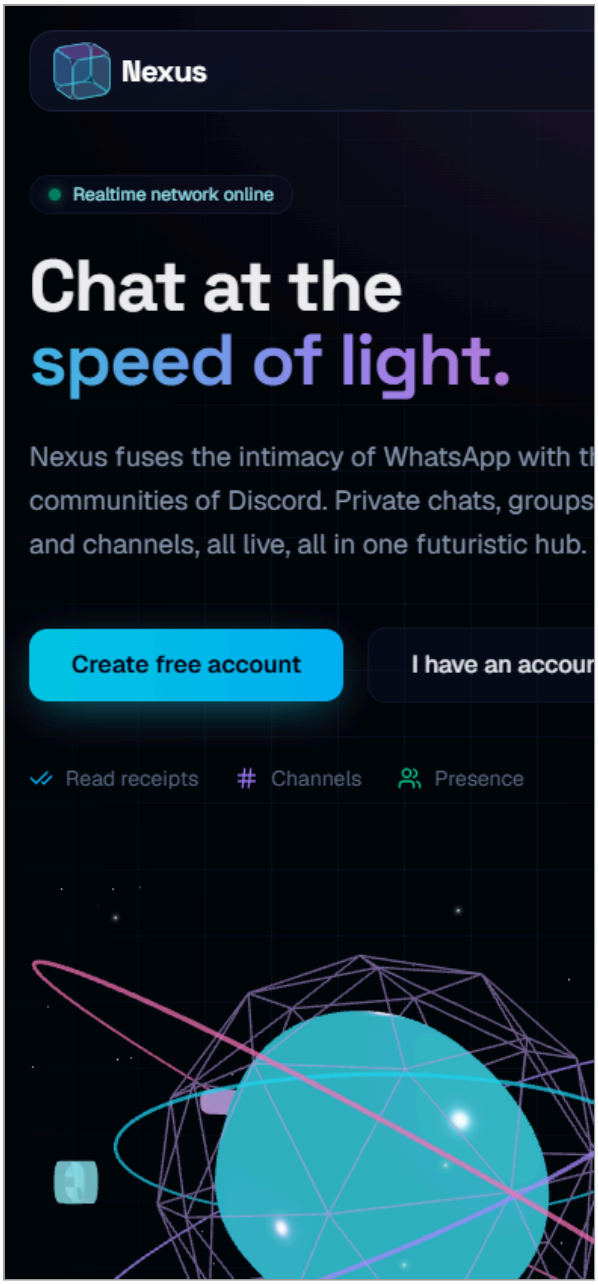


Figure 5: Landing page on a phone (responsive)

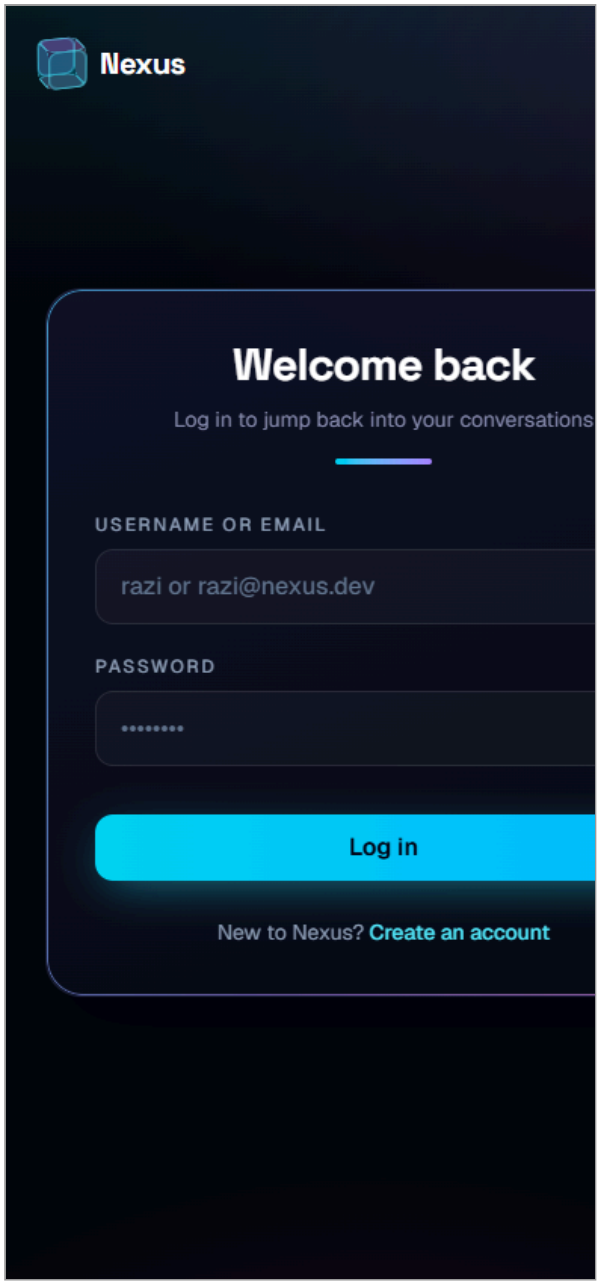


Figure 6: Login page on a phone (responsive)

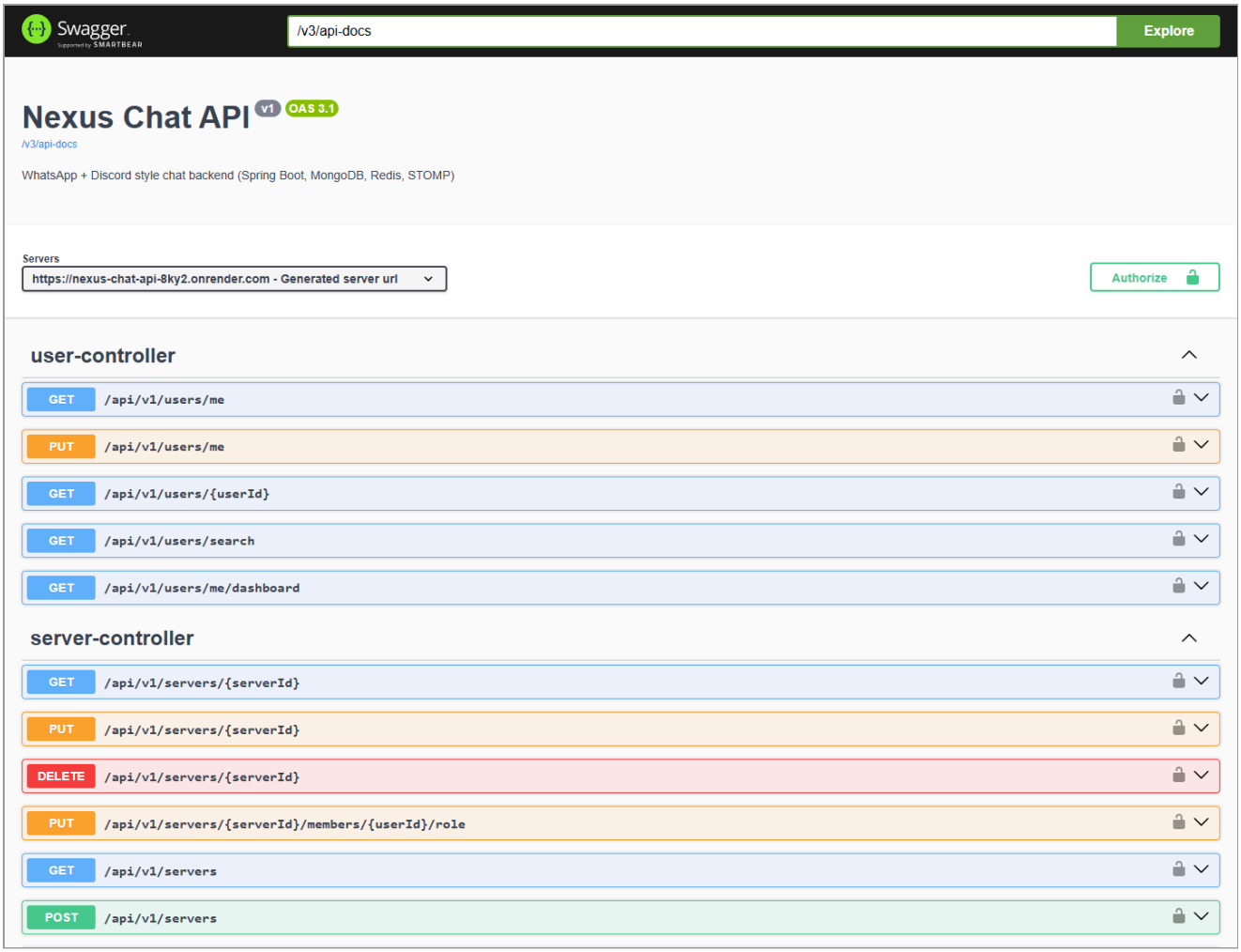


Figure 7: Live API documentation (Swagger UI) of the deployed backend



Figure 8: Live backend response for an unauthenticated request: the custom error envelope with errorCode AUTH_401 and a requestId

6. CONCLUSION

Nexus successfully implements a real-time chat platform that combines WhatsApp-style private and group chats with Discord-style servers and channels. The project shows how Next Generation Databases solve problems that are hard for a single relational database:

- **MongoDB** stores flexible, nested chat data in 10 well-designed collections. Compound, unique and partial indexes make the most common query (the newest messages of a room) fast regardless of data size, and a unique index guarantees that a retried message is never stored twice.
- **Redis** handles data that changes on almost every action (online users, unread badges, rate limits, logged-out tokens) using atomic in-memory operations with automatic expiry, while the application keeps working through a fallback if Redis is unavailable.
- **WebSocket + STOMP** delivers messages, read receipts, typing indicators and notifications instantly.
- Security is handled with BCrypt, JWT and server-side role-based access control, and every error reaches the user in one consistent, friendly format.

The system was verified with 28 automated tests (two real race-condition bugs were found and fixed through them) and deployed for free on Vercel, Render, MongoDB Atlas and Upstash, where it can be used from any laptop or phone.

Learning outcomes

- Designing a NoSQL data model around queries (indexes, denormalisation, cursor pagination) instead of around tables.
- Choosing the right database for each kind of data (polyglot persistence).
- Building realtime, concurrent systems safely: idempotency, atomic counters, handling retries and race conditions.
- Full-stack development and deployment: Spring Boot, Next.js, Docker and cloud services.

Future scope

- Voice and video calls using WebRTC.
- File, image and voice-note sharing using cloud object storage.
- End-to-end encryption for private chats.
- Push notifications on mobile and a native mobile app.
- Scaling to many backend instances with an external message broker (RabbitMQ / Redis Pub-Sub) and MongoDB sharding.
- AI features such as message summaries and smart replies.